

Structure of C Program

Sections of the C Program

Documentation

In a C program, single-line comments can be written using two forward slashes i.e., //, and we can create multi-line comments using /* */. Here, we've used multi-line comments.

```
/**  
 * file: age.c  
 * author: you  
 * description: program to find our age.  
 */
```

Preprocessor Section

All header files are included in this section.

A header file is a file that consists of C declarations that can be used between different files. It helps us in using others' code in our files. A copy of these header files is inserted into your code before compilation.

```
#include <stdio.h>
```

Definition

A preprocessor directive in C is any statement that begins with the "#" symbol. The #define is a preprocessor compiler directive used to create constants. In simple

terms, `#define` basically allows the macro definition, which allows the use of constants in our code.

```
#define BORN 2000
```

We've created a constant `BORN` which is assigned a value of 2000. Generally, uppercase letters are preferred for defining the constants. The above constant `BORN` will be replaced by 2000 throughout our code wherever used.

`#define` is typically used to make a source program easy to modify and compile in different execution environments.

The define statement **does not** end with a semicolon.

Global Declaration

This section includes all global variables, function declarations, and static variables. The variables declared in this section can be used anywhere in the program. They're accessible to all the functions of the program. Hence, they are called global variables.

```
int age();
```

We've declared our `age` function, which takes one integer argument and returns an integer.

Main Function

In the structure of a C program, this section contains the main function of the code. The [C compiler](#) starts execution from the `main()` function. It can use global variables, static

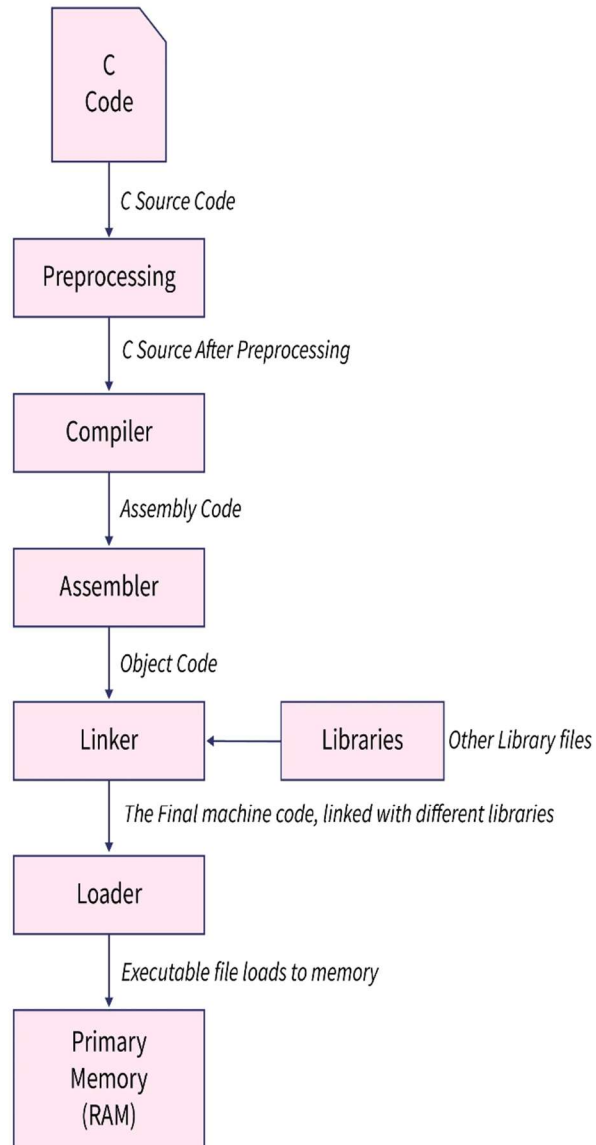
variables, inbuilt functions, and user-defined functions. The return type of the main() function can be void and also not necessarily int.

```
int main()
{
    printf("Age: %d", age);
    return 0;
}
```

Example

```
/*
file: age.c
description: program to find our age.
*/
author: you //Documentation
#include <stdio.h>    //preprocessor directives
#define BORN 2000    //Definition
int age=20; //Global Declaration
int main() //Main Function
{
    printf("Age: %d", age);
    return 0;
```

}



Source Code – This is the code that you have written. This code is sent to the Preprocessors section.

Preprocessing – Preprocessing is the **first step** before actual compilation. It is handled by the **C Preprocessor**, which processes all the **directives** that start with #. Handled by the **C Preprocessor**.

- Deals with all lines that start with # (like #include, #define).
- Inserts the code from header files (like stdio.h) into your file.
- Removes comments.

Compiler – After generating the preprocessed source code, it moves to the compiler and the compiler generates the assembly level code after compiling the whole program.

Assembler – This part takes the assembly level language from compiler and generates the Object code, this code is quite similar to the machine code (set of binary digits).

Linker – Linker is another important part of the compilation process. It takes the object code and link it with other library files, these library files are not the part of our code, but it helps to execute the total program. After linking the Linker generates the final Machine code which is ready to execute.

- Combines your object file with standard libraries (like the printf function from stdio.h).

- This is done by the **linker**.
- The result is a **final executable** file you can run.

Loader – A program, will not be executed until it is not loaded in to Primary memory. Loader helps to load the machine code to RAM and helps to execute it

The structure of a C program, foundational to the basic structure of C programming, is categorized into key sections: Documentation, Definition, Global Declaration, Main() Function. The Main() function is essential in every C program, highlighting what is required in each C program. This organization facilitates readability, modification, and documentation, providing a clear structure of C language.

Data types in C

Each variable in C has an associated data type. It specifies the type of data that the variable can store like integer, character, floating, double, etc

Types	Description	Data Types
Primitive Data Types	Primitive data types are the most basic data types that are used for representing simple values.	int, char, float, double, void

Types	Description	Data Types
Derived Types	The data types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types.	array, pointers, function
User Defined Data Types	The user-defined data types are defined by the user himself.	structure, union, enum

variable in C must be a specified **data type**, and you must use a **format specifier** inside the printf() function to display it

Data Type	Size (64-bit)	Format Specifier	Description
int	4 bytes	%d	Integer numbers
float	4 bytes	%f	Decimal numbers (single)
double	8 bytes	%lf	Decimal numbers (double)
char	1 byte	%c	Single character

Data Type	Size (64-bit)	Format Specifier	Description
void	0 bytes	—	No value / empty type

Length/Type Modifiers

<u>Modifier</u>	<u>Used With</u>	<u>Example</u>	<u>Effect</u>
<u>short</u>	<u>int</u>	<u>short int x;</u>	<u>Smaller integer range (2 bytes)</u>
<u>long</u>	<u>int, double</u>	<u>long int y;</u>	<u>Larger integer (8 bytes on 64-bit)</u>
<u>long long</u>	<u>int</u>	<u>long long int z;</u>	<u>Even larger integer (8 bytes)</u>
<u>unsigned</u>	<u>Integers</u>	<u>unsigned int a;</u>	<u>Only positive values (doubles max)</u>
<u>signed</u>	<u>Integers</u>	<u>signed int b;</u>	<u>Includes negative values (default)</u>

Variables in C

Definition

A **variable** in C is a **named memory location** used to **store a value that can be changed during the execution of a program.**

For example:

```
int age = 20;
```

Here:

- `int` → Data type
- `age` → Variable name
- `20` → Value stored in the variable

The value can be changed:

```
age = 25;
```

Now age contains 25.

Why do we use variables?

Variables are used to **store data temporarily in computer memory** so that we can use and modify that data in a program.

Example:

```
int a = 10;  
int b = 20;  
int sum;
```

```
sum = a + b;
```

Here:

- a stores 10
- b stores 20
- sum stores the result 30

Declaration of a Variable

Before using a variable, we normally declare it.

Syntax:

```
data_type variable_name;
```

Example:

```
int age;  
float salary;  
char grade;
```

Multiple variables can also be declared:

```
int a, b, c;
```

Initialization of a Variable

Giving an initial value to a variable at the time of declaration is called **initialization**.

```
int age = 20;  
float marks = 85.5;  
char grade = 'A';
```

Example Program

Statement: Write a C program to store and display a student's age.

```
#include <stdio.h>  
  
int main() {  
    int age = 20;  
  
    printf("Age = %d", age);  
  
    return 0;  
}
```

Output:

```
Age = 20
```

Variable Value Can Change

```
#include <stdio.h>

int main() {
    int age = 20;

    printf("Before = %d\n", age);

    age = 25;

    printf("After = %d", age);

    return 0;
}
```

Output:

```
Before = 20
After = 25
```

This is the main difference between a **variable** and a **constant**.

Variable	Constant
Value can be changed	Value cannot be changed
<code>int age = 20;</code>	<code>const int age = 20;</code>
Used for changing data	Used for fixed data

Simple definition for exam

A variable is a named memory location that is used to store data, and its value can be changed during program execution.

Constants in C

Definition:

A constant in C is a value that cannot be changed during the execution of a program.

For example:

```
int age = 20;
```

Here, 20 is a **constant** because its value is fixed.

Types of Constants in C

There are mainly **5 types of constants**:

1. Integer Constant

Whole numbers without decimal points.

Examples:

10

-25

0

1000

Example:

```
int age = 20;
```

Here, 20 is an integer constant.

2. Floating-Point Constant

Numbers containing a decimal point.

Examples:

```
3.14  
10.5  
-25.75
```

Example:

```
float pi = 3.14;
```

Here, 3.14 is a floating-point constant.

3. Character Constant

A single character enclosed within **single quotes** ' '.

Examples:

```
'A'  
'b'  
'5'
```

'@'

Example:

```
char grade = 'A';
```

Here, 'A' is a character constant.

'5' is a **character constant**, while 5 is an **integer constant**.

4. String Constant

A group of characters enclosed within **double quotes** "
".

Examples:

```
"Hello"
```

```
"Anupam"
```

```
"Hello World"
```

Example:

```
printf("Hello World");
```

Here, "Hello World" is a string constant.

5. Enumeration Constant

Constants defined using the `enum` keyword.

Statement: Create constants for days of the week.

```
#include <stdio.h>

int main() {
    enum Day {
        MONDAY,
        TUESDAY,
        WEDNESDAY
    };

    printf("%d", MONDAY);

    return 0;
}
```

Here, MONDAY, TUESDAY, and WEDNESDAY are **enumeration constants**.

How to create a constant using `const`

We can use the `const` keyword to make a variable's value unchangeable.

Statement: Write a C program to store a fixed value of PI.

```
#include <stdio.h>

int main() {
    const float PI = 3.14;

    printf("PI = %.2f", PI);

    return 0;
}
```

If we try:

```
PI = 5.0;
```

the compiler will give an **error** because PI is declared as const.

Easy way to remember

Type	Example
Integer	10, -5, 100
Floating	3.14, 10.5
Character	'A', 'x', '5'
String	"Hello", "C Programming"
Enumeration	MONDAY, TUESDAY

Important:

'A' → Character constant

"A" → String constant

Operators in C

Definition:

An **operator** is a symbol that tells the compiler to perform a particular operation on one or more operands.

Example:

a + b

Here + is the **operator**, while a and b are **operands**.

Today we will discuss three important types:

1. **Arithmetic Operators**
2. **Relational Operators**
3. **Logical Operators**

1. Arithmetic Operators

Arithmetic operators are used to perform **mathematical calculations**.

Operator	Meaning	Example	Result
+	Addition	10 + 5	15

Operator	Meaning	Example	Result
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Modulus (remainder)	10 % 3	1

Example

Statement: Write a C program to perform arithmetic operations on two numbers.

```
#include <stdio.h>
int main() {
    int a = 10, b = 3;
    printf("Addition = %d\n", a + b);
    printf("Subtraction = %d\n", a - b);
    printf("Multiplication = %d\n", a * b);
    printf("Division = %d\n", a / b);
    printf("Remainder = %d\n", a % b);
    return 0;
}
```

Output:

```
Addition = 13
Subtraction = 7
Multiplication = 30
Division = 3
Remainder = 1
```

Note: With integers, $10 / 3$ gives 3, not 3.33.

2. Relational Operators

Relational operators are used to **compare two values**.

The result is either:

- $1 \rightarrow \text{True}$
- $0 \rightarrow \text{False}$

Operator	Meaning	Example	Result
>	Greater than	$10 > 5$	1
<	Less than	$10 < 5$	0
>=	Greater than or equal to	$10 \geq 10$	1
<=	Less than or equal to	$5 \leq 10$	1
==	Equal to	$10 == 10$	1
!=	Not equal to	$10 != 5$	1

Example

Statement: Write a C program to compare two numbers.

```
#include <stdio.h>
int main() {
    int a = 10, b = 20;

    printf("%d\n", a > b);
}
```

```
    printf("%d\n", a < b);  
    printf("%d\n", a == b);  
    printf("%d\n", a != b);  
    return 0;  
}
```

Output:

```
0  
1  
0  
1
```

Important: = vs ==

```
a = 10;
```

= means **assignment**.

```
a == 10;
```

== means **comparison/equality**.

3. Logical Operators

Logical operators are used to **combine or reverse conditions**.

There are three logical operators:

Operator	Name	Meaning
& &	Logical AND	Both conditions must be true
	Logical OR	Atleast one condition be true
!	Logical NOT	Reverses the result

Logical AND &&

Both conditions must be true.

Example:

$(10 > 5) \ \&\& \ (20 > 10)$

Both are true $\rightarrow$ Result = 1

Truth Table

A	B	A && B
0	0	0
0	1	0
1	0	0
1	1	1

Logical OR ||

At least **one condition** must be true.

Example:

$(10 > 20) \ || \ (20 > 10)$

First is false, second is true $\rightarrow$ Result = 1

Truth Table

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

Logical NOT !

NOT reverses the result.

$!(10 > 5)$

$10 > 5$ is true (1)

NOT of true $\rightarrow$ false (0)

Truth Table

A	!A
0	1
1	0

Combined Example

Statement: Write a C program to check whether a person is eligible based on age and marks.

```
#include <stdio.h>
int main() {
    int age = 20;
    int marks = 75;

    if (age >= 18 && marks >= 50) {
        printf("Eligible");
    }
    return 0;
}
```

Here:

age >= 18

is a **relational condition**.

marks >= 50

is another **relational condition**.

&&

is a **logical operator** that combines both conditions.

Output:

Eligible

Quick Revision

Arithmetic → Calculation

+ - * / %

Relational → Comparison

> < >= <= == !=

Logical → Conditions combine/reverse

&& || !

Type Casting in C

Definition

Type Casting is the process of **converting a value from one data type to another data type**.

For example, converting an `int` into a `float`.

Syntax

```
(new_data_type) value;
```

Example:

```
float result = (float)10;
```

Here, 10 is temporarily converted from `int` to `float`.

Why is Type Casting Needed?

It is mainly used when we want to perform an operation between **different data types** or want a particular type of result.

Example without Type Casting

Statement: Divide two integers and display the result.

```
#include <stdio.h>

int main() {
    int a = 5;
    int b = 2;
    float result;
    result = a / b;
    printf("Result = %.2f", result);
    return 0;
}
```

Output:

Result = 2.00

Why?

Because a and b are both integers, so:

$$5 / 2 = 2$$

Example with Type Casting

```
#include <stdio.h>

int main() {
    int a = 5;
    int b = 2;
```

```
float result;  
result = (float)a / b;  
printf("Result = %.2f", result);  
return 0;  
}
```

Output:

Result = 2.50

Here:

`(float)a`

converts `a` from `int` to `float`.

So:

5 / 2
↓
5.0 / 2
↓
2.5

Types of Type Conversion

There are two main types:

1. Implicit Type Conversion

The compiler **automatically** converts one data type into another.

Example:

```
int a = 10;  
float b;
```

```
b = a;
```

Here, `int` is automatically converted to `float`.

```
10 → 10.0
```

2. Explicit Type Conversion (Type Casting)

The programmer **manually** converts the data type using the cast operator.

Example:

```
int a = 5;  
int b = 2;
```

```
float result = (float)a / b;
```

Here, `(float)` is explicitly used for conversion.

Important Example

```
int a = 10;  
float b = 3.5;
```

```
float result;
```

```
result = a + b;
```

Here C automatically converts a from int to float.

$10 \rightarrow 10.0$

$10.0 + 3.5 = 13.5$

So:

```
result = 13.5
```

Easy way to remember

Implicit: Compiler converts automatically.

Explicit: Programmer tells the compiler to convert using:

`(data_type)`

Example:

`(float) 5`

`(int) 5.8`

`(char) 65`

Exam definition:

Type casting is the explicit conversion of a value from one data type to another using the cast operator `(data_type)`.